Setting up VS Code for `.ipynb` notebooks with Python 3.13

Prerequisites

Before starting, make sure:

- You have **Python 3.13** installed.
- You have **Visual Studio Code (VS Code)** installed.

Step 1: Install Python 3.13

1. Visit the official Python website: https://www.python.org/downloads/release
2. Download the **Python 3.13.x** installer for your OS.
3. Run the installer and check:
   - Add Python to PATH
   - Install pip (default option)
4. After installation, confirm Python version by running in terminal:

```
python --version
```

   or, if on Unix/Mac:

```
python3 --version
```

Step 2: Install VS Code

1. Download from https://code.visualstudio.com/
2. Install as per your operating system.

Step 3: Install Required VS Code Extensions

In VS Code:

1. Go to the **Extensions** view (`Ctrl + Shift + X`).
2. Search for and install:
   - **Python** (by Microsoft)
   - **Jupyter** (by Microsoft)

These enable full support for `.ipynb` notebooks.

Step 4: Create a New Notebook

1. Open a folder in VS Code where you want to store notebooks.
2. Click `File` → `New File`.
3. Save it with the extension `.ipynb`, e.g., `test_notebook.ipynb`.

Step 5: Select Python 3.13 Interpreter

1. Click the **kernel selector** (top-right of notebook or bottom blue bar).
2. Select your **Python 3.13** interpreter. It may appear like:

```
Python 3.13.0 64-bit ('base': conda) or python3.13
```

3. If not visible, manually locate:
   - Press `Ctrl + Shift + P` → type and select `Python: Select Interpreter`
   - Browse or enter path to `python3.13`

Step 6: Install Jupyter Kernel for Python 3.13

If the kernel isn't available for your Python 3.13 installation:

1. Open terminal (`Ctrl + ``).
2. Run:

```
python -m pip install jupyter ipykernel
python -m ipykernel install --user --name=python313 --display-name "Python 3.13"
```

3. Now go back to the notebook and choose the newly created `"Python 3.13"` kernel.

Step 7: Test a Cell

Try running a simple code cell like:

```
print("Hello from Python 3.13!")
```

It should execute and return the output below the cell.

Troubleshooting

- If you see **"Python not installed"**, make sure Python 3.13 is in your system PATH.
- If Jupyter kernel doesn't show, ensure you've installed `ipykernel` in the correct Python environment.

You're Ready!

You can now:

- Write and run `.ipynb` notebooks in VS Code
- Use Python 3.13 kernel for your analysis
- Enjoy features like variable explorer, interactive plotting, and debugging!

Orientation

- Jupyter Notebooks are a way to share and interactively run code from your browser.
- Notebooks are made of cells
- Each cell contains either code or text.
- Code cells run the code, text cells render the text
- To run a cell, click on the cell and click the Play button or do "shift + Enter" on your keyboard.
- Python runs in all kinds of environments. These instructions are specific to Colab Kernels/Jupyter Notebooks.

Part I - Basic Buiding Blocks

1. Printing values
2. Using comments
3. Variables, assignment, and re-assignment
4. Data Types
5. Mathematical operations in Python
6. Comparison operators
7. Logical operations in Python
8. Conditionals

1. Printing with the `print()` function

```
print("Good Morning, you all!")
print("hello world!")
```

```
Good Morning, you all!
hello world!
```

```
print(2+2)
```

```
4
```

```
2 + 2
```

```
4
```

```python
# assigning a value to a variable
today = "Today is a great day to learn"
print(today)
```

```
Today is a great day to learn
```

```python
print("Use the print command")
print("To print")
print("Prints each and every line")
```

```
Use the print command
To print
Prints each and every line
```

```python
"Notebook cells automatically print the last line of code"
```

```
'Notebook cells automatically print the last line of code'
```

```python
# run this cell and the number 5 will be printed
5
```

```
5
```

```python
5 * 2
```

```
10
```

```python
b = 5*2
b
```

```
10
```

```python
print("Multiple lines")
print("do not print automatically")
"only the last line"
```

```
Multiple lines
do not print automatically
```

```
'only the last line'
```

```python
'I am writing Python with you all right now'
```

```
'I am writing Python with you all right now'
```

```python
print("That means")
print("The print command is important.")
print("When we want to print each line")
```

```
That means
The print command is important.
When we want to print each line
```

```
# Exercise:
# Print your name in this cell. Be sure to use quotation marks around
the letters.
print("Tanvir")
```

```
Tanvir
```

```
# Exercise: Print your favorite number in this cell
print(7)
```

```
7
```

2. Comments are how we notate our code with descriptive text

- Python comments are created by the hashtag symbol.
- The hashtag "comments out" everything to the right of the hashtag.
- Comments exist in the code but do not execute
- Commenting your code is an industry best practice
- A hashtag at the beginning of a line "comments out

```
'''sdsfgdsfgd


sdfsdafsdaf


sdfsdaf'''
```

```
'sdsfgdsfgd\n\n\nsdfsdafsdaf\n\n\nsdfsdaf'
```

```
# This is a comment. Run this cell. Do you see anything? Why or why
not?

# First, run this cell.
# Then delete the hashtag at the beginning of the last line.
# Last, run the cell again.
print("This is a 'commented out' print command")
print('This is a "commented out" print command')
```

```
This is a 'commented out' print command
This is a "commented out" print command
```

3. Variables point to values and give them a name

- "Assigning a variable" means to bind value to a variable
- Variables allow us to name values
- Variables hold onto values for later.
- Variables hold their assigned value(s) until
- Applications use the same variable to hold different values for different users

```python
# Variables hold values that we use later in the program by name.
# "Assigning a variable"
# Single = sign is called the assignment operator
message = "Hello, Everybody!"
print(message)
```

```
Hello, Everybody!
```

```python
favorite_quote = "If you can't solve a problem, then there is an
easier problem you can solve:  find it."
print(favorite_quote)
```

```
If you can't solve a problem, then there is an easier problem you can
solve:  find it.
```

```python
# Variables are assigned to point to values.
favorite_food = "burger"
print("My favorite food is", favorite_food)

print(f"My favorite food is {favorite_food}")
print(f"My favorite food is {favorite_food}")
print("I am",18*2,"years old")
print("This is a line",end="")
print("This is another line",end="")
```

```
My favorite food is burger
My favorite food is burger
My favorite food is burger
I am 36 years old
This is a lineThis is another line
```

```python
# Variables can also be re-assigned to point to different values.
favorite_food = "pizza"
print("I changed my mind. My favorite food is actually",
favorite_food)
```

```
I changed my mind. My favorite food is actually pizza
```

```python
# Exercise: Reassign the favorite_food variable to hold your own
favorite food,
#then print the variable.
```

4. Data Types

Each piece of data has two fundamental characteristics:

- The value of the data itself.
- Every value has a data type which explains what kind of data the value is.
- For example, the integer 2 has the value of 2 and the data type is integer.
- The first data-types to learn:
  - None

- Boolean
- Integer
- Float
- String
- List
- Function
- Dictionary

```
# The "None" data type represents the absence of a value. This is
called "null" in some other programming languages
type(None)

NoneType

type(True)

bool

type(False)

bool

# Numbers without a decimal are integers. Integers can be negative or
positive.
type(-99)

int

type(2.3)

float

.5 * .5

0.25

x=23
type(x)

int

# True and False are the only two Boolean values.

print(True)
print(False)

True
False
```

5. Mathematical Operations in Python

- Computers are not intelligent.
- Programming languages need to have the ability to perform mathematical operations.

- Mathematical operators in Python evaluate and provide
- Order of operations in Python follows PEMDAS:
  - Parentheses
  - Exponents
  - Multiplication
  - Division
  - Addition
  - Subtraction

Operators operate on values and produce values

Let's look at some math operators in Python

- `+` adds two numbers together
- `-` subtracts the second number from the first
- `*` multiplies both numbers and returns the product
- `/` divides the first number by the second
- `%` provides the remainder of dividing the first number by the second. (called the modulo operator)
- `**` is the exponentiation operator. 2 raised to the 3rd power is `2**3`

```
a = 10
b = 2
print(a+b)
sum = a+b
print(sum)

print(10+2)
print("a+b =",a+b)
print("a+b = ",a+b)
print(f"a+b = {a+b}")
print(f"a+b ={a+b}")
print(f"{a+b=}")
print(f"{a+b = }")
# Output a = 12

12
12
12
a+b = 12
a+b =  12
a+b = 12
a+b =12
a+b=12
a+b = 12

print(f"a-b = {a-b}")

a-b = 8
```

```
print(f"a*b = {a*b}")
```

```
a*b = 20
```

```
print(f"a/b = {a/b}")
```

```
a/b = 5.0
```

```
print(f"a%b = {a%b}")
```

```
a%b = 0
```

```
print(f"a^b = a**b = {a**b}")
```

```
a^b = a**b = 100
```

```
print(f"9**-2 = {9**(-2)}")
```

```
9**-2 = 0.012345679012345678
```

```
# Demonstration Examples (Different Numbers)

# Example: Add numbers from 11 through 20
sum_example = 11 + 12 + 13 + 14 + 15 + 16 + 17 + 18 + 19 + 20
print("Sum Example:", sum_example)

print("Sum Example:", 11 + 12 + 13 + 14 + 15 + 16 + 17 + 18 + 19 + 20)
```

```
Sum Example: 155
Sum Example: 155
```

```
# Example: Subtract 45 from 100
subtraction_example = 100 - 45
print("Subtraction Example:", subtraction_example)

# Example: Multiply 1.5 by 3.2 and check data type
float_multiplication = 1.5 * 3.2
print("Float Multiplication Example (1.5 * 3.2):",
float_multiplication)
print("Data type:", type(float_multiplication))

# Example: Multiply 6 by 9
multiplication_example = 6 * 9
print("Multiplication Example (6 * 9):", multiplication_example)

# Example: Data type of 12 * 4
type_example = 12 * 4
print("Data type of 12 * 4:", type(type_example))

# Example: Divide 5 by 8
division_example = 5 / 8
```

```python
print("Division Example (5 / 8):", division_example)
print("Data type:", type(division_example))
```

```
Subtraction Example: 55
Float Multiplication Example (1.5 * 3.2): 4.800000000000001
Data type: <class 'float'>
Multiplication Example (6 * 9): 54
Data type of 12 * 4: <class 'int'>
Division Example (5 / 8): 0.625
Data type: <class 'float'>
```

```python
# print(10/0)

# Example: Division by zero handling
try:
    error_example = 10 / 0
except ZeroDivisionError as e:
    print("Division by Zero Error Example:", e)
```

```
Division by Zero Error Example: division by zero
```

```python
# Example: Modulo operations
print("13 % 4 =", 13 % 4)
print("10 % 5 =", 10 % 5)
print("15 % 6 =", 15 % 6)

# Example: Exponentiation
print("3 ** 4 =", 3 ** 4)
print("5 ** -2 =", 5 ** -2)
print("99 ** 0 =", 99 ** 0)

# Example: Average of three new values
x, y, z = 40, 50, 60
avg_example = (x + y + z) / 3
print("Average Example:", avg_example)
```

```
13 % 4 = 1
10 % 5 = 0
15 % 6 = 3
3 ** 4 = 81
5 ** -2 = 0.04
99 ** 0 = 1
Average Example: 50.0
```

Exercise: Add 1 plus 2 plus 3 plus 4 plus 5 plus 6 plus 7 plus 8 plus 9 plus 10

Exercise: Subtract 23 from 55

Exercise: Multiply 2.3 by 4.4. What's the data-type that this returns?

Exercise: Multiply 3 by 5.

Exercise: Print out the data-type of the result of multiplying 11 * 7

Exercise: Divide 1 by 2. What happens? What data type is returned?

Exercise: Divide 5 by 0. What happens? Why do you think that is?

Exercise: Use the modulo operator to obtain the remainder of dividing 5 by 2

5 % 2

Exercise: Use the modulo operator to obtain the remainder of dividing 8 by 2

Exercise: Use the modulo operator to obtain the remainder of dividing 7 by 2

Exercise: Use the exponent operator to raise 2 to the 3rd power

Exercise: Use the exponent operator to raise 2 to the negative 1st power

Exercise: Use the exponent operator to raise 123 to the 0th power

Exercise: Use what you have learned to determine the average of the following variables

```
# Exercise: Add 1 plus 2 plus 3 plus 4 plus 5 plus 6 plus 7 plus 8
plus 9 plus 10
# Exercise: Subtract 23 from 55
# Exercise: Multiply 2.3 by 4.4. What's the data-type that this
returns?
# Exercise: Multiply 3 by 5.
# Exercise: Print out the data-type of the result of multiplying 11 *
7
```

```
# Exercise: Divide 1 by 2. What happens? What data type is returned?

# Exercise: Divide 5 by 0. What happens? Why do you think that is?

# Exercise: Use the modulo operator to obtain the remainder of
dividing 5 by 2
# 5 % 2
# Exercise: Use the modulo operator to obtain the remainder of
dividing 8 by 2

# Exercise: Use the modulo operator to obtain the remainder of
dividing 7 by 2

# Exercise: Use the exponent operator to raise 2 to the 3rd power

# Exercise: Use the exponent operator to raise 2 to the negative 1st
power

# Exercise: Use the exponent operator to raise 123 to the 0th power

# Exercise: Use what you have learned to determine the average of the
following variables
```

6. Comparison Operators

Python Comparison Operators compare two values and return a True or False

- `==` compares two values to see if they are the same value.
- `!=` asks if the values on each side of the operator are **not** the same.]
- `<` and `<=` are less than and less than or equal to
- `>` and `>=` are greater than and greater than or equal to, respectively.

Let's look at some concrete examples of using comparison operators and values that return a True or a False.

- `2 < 3` returns `True` because it's a true statement that two is less than three
- `2 <= 2` returns `True` because two is less than or equal to two
- `2 < 2` returns `False` because two is not less than two
- `4 <= 2` returns `False` because four is not less than or equal to two
- `10 < 5` returns `False` because 10 is not less than 5
- `10 > 5` returns `True` because ten is greater than 5
- `2 == 2` returns `True` because the number two is equal to the number two
- `2 == 3` returns `False` because two is not three
- `3 != 2` returns `True` because three is not equal to two.
- `4 in [1, 2, 3, 4]` returns `True` because 4 exists in the list `[1, 2, 3, 4]`

- `"b" in "banana"` returns `True` because the string "b" exists inside the string `"banana"`

If we have `x = 3` followed by `result = x > 0` on the next line, the result variable will hold `True` because 3 is greater than 0.

Comparison Operators ask the following questions

- Are these two things equal?
- Are these two things not equal?
- Is A greather than B?
- Is A greater than or equal to B?
- Is A less than B?
- Is A less than or equal to B?
- Does A exist in a list or collection of values named B

```
print("5 > 4 is", 5 > 4)
print("1 > 0 is", 1 > 0)
# Comparison operators give us back a True or False
print(5 == 5)
print(-4 + 4 == 0)
# False because they're not the same data type
5 == "5"
print("Hello" == "Goodbye")
print("Hello" != "Goodbye") # != means "not equal to"
# = assignment# False because they're not the same data type
5 == "5"
```

```
5 > 4 is True
1 > 0 is True
True
True
False
True

False
```

```
# Exercise: assign the variable favorite_number to hold your favorite
number

# Exercise: Write the Python code necessary to answer if your favorite
number is less than 100

# Exercise: Write the code to determine if your favorite number is
greater than 0

# Exercise: Write the code to determine if your favorite number is
five

# Exercise: Write the code to determine if your favorite number is
greater than or equal to 3
```

```
# Exercise: Write the code to determine if your favorite number is greater than or equal to -100

favorite_number = 7

# Before running this code, consider if this will return True or False, given your favorite_number variable's value


favorite_number < 0 and favorite_number > 10
```

```
False
```

```
# Before running this code, consider if this will return True or False, given your favorite_number variable's value

favorite_number < 10 or favorite_number > 20
```

```
True
```

7. Logical Operators

Evaluating what's True in Python

- Most common logical operations are:
  - And
  - Or
  - Not
- For a reminder, see Conjunction Junction from School House Rock

```
not True
```

```
False
```

```
not False
```

```
True
```

```
not not True
```

```
True
```

```
not not False
```

```
False
```

```
# Truth table for AND
print("Truth Table for AND logic")
print("True and True is:", True and True)
print("True and False is:", True and False)
print("False and True is:", False and True)
print("False and False is:", False and False)
```

```
Truth Table for AND logic
True and True is: True
True and False is: False
False and True is: False
False and False is: False
```

```python
# Truth table for OR
print("Truth Table for OR logic")
print("True or True is:", True or True)
print("True or False is:", True or False)
print("False or True is:", False or True)
print("False or False is:", False or False)
```

```
Truth Table for OR logic
True or True is: True
True or False is: True
False or True is: True
False or False is: False
```

```python
# Boolean values assigned to variables
right_here_now = True
learning_python = True
on_the_moon = False

# Ands need all values to be true for the entire expression to be True
print(right_here_now and on_the_moon)


# Read the line of code below and think about the result.
# An OR statement only needs a single True for the entire expression
to be True
print(learning_python or on_the_moon)
```

```
False
True
```

8. Conditional Logic with Conditionals

- Using `if`
- Using `if, else`
- Using `if, elseif, else`

The "if" statement is how we write "If this then that"

- If the code evalutates to `True` then Python runs the code that's indented after the "if" line

```python
if False:
    print("So glad you made it today!")
    print("Wherever you go, there you are.")
elif True:
```

```
  print("Now you are in this cell")
else:
    print("You are not right here now.")
```

```
Now you are in this cell
```

```
right_here_now = False

if right_here_now == False:
    print("So glad you made it today!")
    print("Wherever you go, there you are.")
elif True:
  print("Now you are in this cell")
else:
    print("You are not right here now.")
```

```
So glad you made it today!
Wherever you go, there you are.
```

```
# the "if" evaluates the code to the right to see if it returns True
or False.
# If the code returns `True`, then we run the code that's indented
below the if.

right_here_now = False
right_2nd_now = True

if (right_here_now):
    print("So glad you made it today!")
    print("Wherever you go, there you are.")
elif(right_2nd_now):
  print("Now you are in this cell")
else:
    print("You are not right here now.")

print("Unindented means the if/else is over, and we're back to our
regularly scheduled programming.")
```

```
Now you are in this cell
Unindented means the if/else is over, and we're back to our regularly
scheduled programming.
```

```
# Before running this cell, think about and predict the outcome.
# the "if" evaluates the code to the right to see if it returns True
or False.
# If true, then we run the code that's indented below the if.

on_the_moon = False

if on_the_moon:
    print("You are currently on a lunar base.")
```

```
else:
    print("You are not on the moon!")
```

```
You are not on the moon!
```

```
# Before running this cell, think about and predict the outcome.

if right_here_now and learning_python:
    print("We are right here now and learning python.")

# Before running this cell, think about and predict the outcome.
if right_here_now and learning_python and on_the_moon:
    print("You must be magic since you're on the moon and here and
learning Python all at the same time.")

# Before running this cell, think about and predict the outcome.
if right_here_now or on_the_moon:
    print("How many True values does it take with an OR for the
entirety to be True?")


# Before running this cell, think about and predict the outcome.
# Carefully consider the parentheses...
if learning_python and (right_here_now or on_the_moon):
    print("In addition to learning python, you are either right here
now or on the moon")


if True and False or True:
    print("In addition to learning python, you are either right here
now or on the moon")

if True and (False or True):
    print("In addition to learning python, you are either right here
now or on the moon")
```

```
In addition to learning python, you are either right here now or on
the moon
In addition to learning python, you are either right here now or on
the moon
```

```
# Exercise: Create a variable called is_raining and assign it True or
False depending on if it's raining right now

# Exercise: Create a variable named will_rain_later and assign a value
of True or False if it may rain later today

# Exercise: Create an "if" condition that checks if it's raining now
or later.
# If it is going to rain now or later, then print out "I'll bring an
umbrella"
```

```
x = 10
if x>0:
    print(x, "is positive")
elif x<0:
    print(x, "is Negative")
else:
    pass
    # print(x, "is zero")
```

```
10 is positive
```

WHILE and FOR LOOPS

We will learn the most basic while and for loops

```
# Stay in loop if condition( i is not equal 5) is true
i = 0
while i != 5 :
    print('i is: ',i)
    i +=1
    # i = i+1
print(i,' is equal to 5')
```

```
i is:  0
i is:  1
i is:  2
i is:  3
i is:  4
5  is equal to 5
```

heading :hash text

heading 2

list

- item 1
- item 2

markdown text

`code`

bold text

```
print("hello")
```

```python
# Stay in loop if condition( i is not equal 5) is true
lis = [1,2,3,4,5]
for i in lis:
    print('i is: ',i)
print('Loop is over')

# Enumerate index and value of list
# index : value = 0:1, 1:2, 2:3, 3:4, 4:5
for index, value in enumerate(lis):
    print(index," : ",value)
print('End of loop')
```

```
i is:  1
i is:  2
i is:  3
i is:  4
i is:  5
Loop is over
0  :  1
1  :  2
2  :  3
3  :  4
4  :  5
End of loop
```

```python
# For dictionaries
# We can use for loop to achive key and value of dictionary. We learnt
key and value at dictionary part.
dictionary = {'spain':'madrid','france':'paris'}
for key,value in dictionary.items():
    print(key," : ",value)
print('')

# For pandas we can achieve index and value
#for index,value in data[['Attack']][0:1].iterrows():
 #   print(index," : ",value)
```

```
spain  :  madrid
france  :  paris
```

Part II - Lists, Dictionaries, and Functions

- With programming and analysis, we work with lots of lists of numbers.
- The primary data type in Python for holding lists of numbers is called a list. It's like an array in other programming languages.

Lists are ordered lists of values.

- Created with square brackets

- Lists can hold any value(s)
- Lists can hold lists.
- Think of a list as a column of values in a spreadsheet (1 dimensional list)
- Think of a list of lists as an the entire spreadsheet (2 dimensional list)
- We can have lists of lists of lists, and that's like having a workbook of spreadsheets (3 dimensions)
- There are other types of sequences and collections in Python, but lists are a solid start!

```
# Lists are created with square brackets. Each item on the list is
separated by a comma. Lists can hold any data type
names = ["Rahim", "Karim", "Jasim"]
print(names)
```

```
['Rahim', 'Karim', 'Jasim']
```

```
# Lists have built-in functions
names.append("Shamim") # Don't forget Ringo

names
```

```
['Rahim', 'Karim', 'Jasim', 'Shamim']
```

```
# .append and .reverse are some of the built-in list functions
names.reverse()
print(names)
```

```
['Shamim', 'Jasim', 'Karim', 'Rahim']
```

```
names
```

```
['Shamim', 'Jasim', 'Karim', 'Rahim']
```

```
# We can also use lists to hold numbers
odds = [1, 3, 5, 7, 9, 11, 13, 17, 19, 21]
odds
```

```
[1, 3, 5, 7, 9, 11, 13, 17, 19, 21]
```

```
# Initial list
my_list = [2, 3, 4]

# Prepend: Insert at the 0th index
my_list.insert(3, 9)

# Output
print("Final list:", my_list)
```

```
Final list: [2, 3, 4, 9]
```

```
# for variable in list:
# For each number in the list of odds, do something
# thing we do each time through the loop IS the code indented within
```

```
that loop
# for new_variable in list_variable

a = [1, 3, 5, 7, 9, 11, 13, 17, 19, 21]

for i in a:
    print(i)
    print("The loop is going")
print("The Loop is complete.")
```

```
1
The loop is going
3
The loop is going
5
The loop is going
7
The loop is going
9
The loop is going
11
The loop is going
13
The loop is going
17
The loop is going
19
The loop is going
21
The loop is going
The Loop is complete.
```

```
odds = [1,3,5,7,9]

for number in odds:
    print(number + 10)
```

```
11
13
15
17
19
```

Functions

There are 2 kinds of functions in Python: built-in and user defined functions.

Some built in functions:

- type, print, len, sum, abs, range, list, min, max, round, etc...

- For a complete list of build in functions, see https://docs.python.org/3/library/functions.html

```
sum = 10  # □ This overrides the built-in function!

del sum  # □ This removes the user-defined variable

numbers = [2, 3, 5, 7]
sum(numbers)
```

```
17
```

Avoid using variable names that match built-in functions like sum, list, min, max, etc. It's a common beginner mistake that can silently break code. You may see error ie.

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[168], line 3
      1 # sum is a built in function
      2 numbers = [2, 3, 5, 7]
----> 3 sum(numbers)

TypeError: 'int' object is not callable
```

```
# sum is a built in function
numbers = [2, 3, 5, 7]
sum(numbers)
```

```
17
```

```
# len is a built-in function short for length
len(numbers)
```

```
4
```

```
# calculate the average of a list of numbers
sum(numbers) / len(numbers)
```

```
4.25
```

```
# The len function also works on strings
programming_language = "Python"
print("The number of letters in Python is", len(programming_language))
```

```
The number of letters in Python is 6
```

Let's Create Some Example Functions

Imagine the exercise asks us to write a function called is_five that takes in a variable and returns True/False if that variable is the number five. The function would look like this:

```
def is_five(x):
    result = x == 5
    return result
```

Or we could write this a little more succinctly (since returning the result is the same as returning the expression assigned to the result variable)

```
def is_five(x):
    return x == 5
```

The result of this function is entirely dependent on the input values we send into the function. For example:

- `is_five(10)` will return `False`
- `is_five(-5)` returns `False` because -5 is not 5.
- `is_five("hello")` returns `False` because the string "hello" certainly isn't the number 5.
- `is_five(5)` returns `True` because 5 == 5 evaluates to True.

Notice how this function is returning the result of a comparison operation and that the comparison operator will give us a True or a False, depending on the value contained in the input variable sent into the function.

If the exercise was to make a function named is_more_than_three, we could define the function in the following way:

```
def is_more_than_three(x):
    return x > 3
```

Notice that we're returning the result of x > 3, which can change with different inputs. For example:

- `is_more_than_three(10)` returns `True`
- `is_more_than_three(30)` returns `True`
- `is_more_than_three(3)` returns `False` (because 3 is not greater than 3)
- `is_more_than_three(3.025)` returns `True` because 3.023 > 3 returns True.
- `is_more_than_three(-23)` returns `False` b/c negative 23 is less than 3.

```
# Exercise: Use the examples above to write a function called
is_greater_than_1 that takes in an input and returns True/False if
that number is greater than 1

# Exercise: Create an "if" condition that checks if it's raining now
or later.
# If it is going to rain now or later, then print out "I'll bring an
umbrella"
```

```python
# Exercise: Use the examples above to write a function called
is_less_than_100 that takes in an input and returns True/False if that
number is less than 100

def printPi(x=0):
    x = 3.14159
    print(x)
    return x

a = printPi()
print(a)
```

```
3.14159
3.14159
```

```python
# how to define our own functions
# def function_name(variable_name_for_the_input):
#      output = guts of the function
#      return output
def is_even(number):
    remainder = number % 2 # the % operator returns the remainder of
integer division of the number on the left by the number on the right
    if remainder == 0: # even numbers have no remainder
        return True
    else:
        return False

# calling a function means to run it
print(is_even(2))
print(is_even(4))
print(is_even(2.3))
print(is_even(3))
```

```
True
True
False
False
```

```python
# lots of built in functions work on sequences
names = ['Rahim', 'Karim', 'Jasim', 'Shamim']

print("The number of items on the list is", len(names))
```

```
The number of items on the list is 4
```

```python
numbers = [2, 3, 5, 7]
print("The highest number on the list is", max(numbers))
print("The smallest number on the list is", min(numbers))
```

```
The highest number on the list is 7
The smallest number on the list is 2
```

```
# def keyword to define, name_of_function, parentheses for parameters
def square(number):
    return number * number # return is the output we hand back to the
code that called the function (calling == execute == run)

square(3)
```

```
9
```

```
square(4)
```

```
16
```

```
# How to average a list of numbers.
numbers = [2, 3, 5, 7]
total = sum(numbers)
average = total / len(numbers)
average
```

```
4.25
```

```
def average(numbers):
    total = sum(numbers)
    average = total / len(numbers)
    return average

print(average([1, 2, 3]))
print(average([1, 2]))
```

```
2.0
1.5
```

```
# Max returns the longest string
print(max(["a", "aa", "aaa"]))
print(min(["a", "aa", "aaa"]))
```

```
aaa
a
```

Introducing Dictionaries

- Dictionaries are like "labeled lists"
- Lists hold values in order and have a numeric index.
- Dictionaries hold values and call them by a string index.
- Dictionaries in the real world are like menus or nutritional facts tables

```
workshop = {
    "name": "Introduction to Python",
    "instructor": "Ryan",
    "platform": "Kaggle",
    "Source": "Internet"
}
```

```python
print(workshop)
```

```
{'name': 'Introduction to Python', 'instructor': 'Ryan', 'platform': 'Kaggle', 'Source': 'Internet'}
```

```python
workshop = {
    "name": "Introduction to Python and Java",
    "instructor": "Ryan and SKP",
    "platform": "Kaggle and JDK"
}
print(workshop)
```

```
{'name': 'Introduction to Python and Java', 'instructor': 'Ryan and SKP', 'platform': 'Kaggle and JDK'}
```

```python
# We use the variable that holds the dictionary followed by square brackets (like a list), with the string holding the key name.
workshop["name"]
```

```
'Introduction to Python and Java'
```

```python
workshop["platform"]
```

```
'Kaggle and JDK'
```

```python
# To add new values to a dictionary, use a new key name
workshop["programming_language"] = "Python"
workshop["language_version"] = 3.6

workshop
```

```
{'name': 'Introduction to Python and Java',
 'instructor': 'Ryan and SKP',
 'platform': 'Kaggle and JDK',
 'programming_language': 'Python',
 'language_version': 3.6}
```

```python
# Exercise
# Add a new key=>value pair to the "workshop" dictionary variable.
# The key should be called "my_name" and the value should be a string holding your name


# Exercise
# Exercise: Create an "if" condition that checks if it's raining now or later.
# If it is going to rain now or later, then print out "I'll bring an umbrella"

# Exercise: Use the examples above to write a function called is_greater_than_1 that takes in an input and returns True/False if that number is greater than 1
```

```
# Make a new variable called preferences.
# Set up a key for "favorite_number" and assign a value
# Setup a key for "favorite_color" and assign a string with your
favorite number
```

Numpy

Exercise 1: Array Creation and Indexing

```
import numpy as np
# Exercise 1
arr1 = np.array([1, 2, 3, 4, 5])
print("arr1:")
print(arr1)
print("Shape of arr1:", arr1.shape)
print("Dimensions of arr1:", arr1.ndim)
print("Second element of arr1:", arr1[1])
print("Last element in arr1:", arr1[-1])
print(f"arr1[0]={arr1[0]}")
print(f"{arr1[:4]=}")
print(f"{arr1[2:]=}")
print(f"{arr1[2:5]=}")
print(f"{arr1[:]=}")
```

```
arr1:
[1 2 3 4 5]
Shape of arr1: (5,)
Dimensions of arr1: 1
Second element of arr1: 2
Last element in arr1: 5
arr1[0]=1
arr1[:4]=array([1, 2, 3, 4])
arr1[2:]=array([3, 4, 5])
arr1[2:5]=array([3, 4, 5])
arr1[:]=array([1, 2, 3, 4, 5])
```

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])
print("arr2:")
print(arr2)
print("Shape of arr2:", arr2.shape)
print("Dimensions of arr2:", arr2.ndim)
print("Element in the second row and third column of arr2:", arr2[1,
2])
print("Last row in arr2:", arr2[-1])
print("All elements in 2nd column of arr2:", arr2[:, 1])
```

```
arr2:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape of arr2: (3, 3)
Dimensions of arr2: 2
Element in the second row and third column of arr2: 6
Last row in arr2: [7 8 9]
All elements in 2nd column of arr2: [2 5 8]
```

```
arr2 = np.array([[1, 2, 3, 6],
                 [4, 5, 6, 8],
                 [7, 8, 9, 76]])

print("Shape of arr2:", arr2.shape)
print("Dimensions of arr2:", arr2.ndim)
```

```
Shape of arr2: (3, 4)
Dimensions of arr2: 2
```

Exercise 2: Basic Operations

```
a = [1, 2, 3, 4, 5]
b = [10, 20, 30, 40, 50]
result = []
for i in range(len(a)):
    result.append(a[i]+b[i])
print(result)
```

```
[11, 22, 33, 44, 55]
```

```
import numpy as np
# Exercise 2
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])
c = a + b
d = a - b
e = a * 2
f = np.dot(a, b)
print("a + b =", c)
print("a - b =", d)
print("a * 2 =", e)
print("Dot product of a and b:", f)
```

```
a + b = [11 22 33 44 55]
a - b = [ -9 -18 -27 -36 -45]
a * 2 = [ 2  4  6  8 10]
Dot product of a and b: 550
```

Exercise 3: Universal Functions

```
import numpy as np

# Exercise 3
grades = np.array([90, 85, 78, 92, 88])

sqrt_grades = np.sqrt(grades)
mean_grade = np.mean(grades)
max_grade = np.max(grades)
sorted_indices = np.argsort(grades)

print("Square root of grades:", sqrt_grades)
print("Mean grade:", mean_grade)
print("Maximum grade:", max_grade)
print(grades)
print("Indices that would sort grades in ascending order:",
sorted_indices)

print("Minimum grade:", np.min(grades))
print("Standard deviation of grades:", np.std(grades))
```

```
Square root of grades: [9.48683298 9.21954446 8.83176087 9.59166305
9.38083152]
Mean grade: 86.6
Maximum grade: 92
[90 85 78 92 88]
Indices that would sort grades in ascending order: [2 1 4 0 3]
Minimum grade: 78
Standard deviation of grades: 4.882622246293481
```

Exercise 4: Array Slicing and Filtering

```
import numpy as np

# Exercise 4
numbers = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

sliced_numbers = numbers[0:1]
# sliced_numbers = numbers[2:7]
every_second_element = numbers[::2]
filtered_numbers = numbers[numbers > 50]

print(f"{numbers = }")
print(f"\nSecond to sixth element: \n{numbers[2:7] = }")
print(f"\nLast three elements: \n{numbers[-3:] = }")

print("\nSliced numbers:", sliced_numbers)
print("\nEvery second element:", every_second_element)
```

```
print("Filtered numbers (numbers > 50):", filtered_numbers)
print(f"{numbers[(numbers>=50) & (numbers<=90)] = }")
```

```
numbers = array([ 10,  20,  30,  40,  50,  60,  70,  80,  90, 100])

Second to sixth element:
numbers[2:7] = array([30, 40, 50, 60, 70])

Last three elements:
numbers[-3:] = array([ 80,  90, 100])

Sliced numbers: [10]

Every second element: [10 30 50 70 90]
Filtered numbers (numbers > 50): [ 60  70  80  90 100]
numbers[(numbers>=50) & (numbers<=90)] = array([50, 60, 70, 80, 90])
```

Exercise 5: Common Data Mining Tasks

```
import numpy as np

# Exercise 5
data = np.random.randint(0, 100, size=(5, 3))

mean_data = np.mean(data)
median_data = np.median(data)
std_data = np.std(data, axis=None)

row_means = np.mean(data, axis=1)
column_max = np.max(data, axis=0)

# Normalize the data
normalized_data = (data - mean_data) / std_data

print("Data:")
print(data)

print("Mean of the entire data:", mean_data)
print("Median of the entire data:", median_data)
print("Standard deviation of the entire data:", std_data)

print("Row means:")
print(row_means)

print("Maximum value of each column:")
print(column_max)

print("Normalized data:")
print(normalized_data)
```

```
Data:
[[91 19 36]
 [70 85 68]
 [89 13 17]
 [39 68 71]
 [99 88 25]]
Mean of the entire data: 58.53333333333333
Median of the entire data: 68.0
Standard deviation of the entire data: 29.472850029966377
Row means:
[48.66666667 74.33333333 39.66666667 59.33333333 70.66666667]
Maximum value of each column:
[99 88 71]
Normalized data:
[[ 1.1015788  -1.34134749 -0.76454545]
 [ 0.38905863  0.89800161  0.32119957]
 [ 1.03371973 -1.54492468 -1.40920655]
 [-0.66275685  0.32119957  0.42298816]
 [ 1.37301505  0.9997902  -1.1377703 ]]
```

```python
min_val = np.min(data)
max_val = np.max(data)
min_max_norm = (data - min_val) / (max_val - min_val)
print("Min-Max Normalized Data:")
print(min_max_norm)
```

```
Min-Max Normalized Data:
[[0.90697674 0.06976744 0.26744186]
 [0.6627907  0.8372093  0.63953488]
 [0.88372093 0.         0.04651163]
 [0.30232558 0.63953488 0.6744186 ]
 [1.         0.87209302 0.13953488]]
```

Exercise 6: Loading and Saving Data

```python
import numpy as np

# Exercise 6
matrix = np.random.rand(4, 4)

# Save the matrix to a file
np.save('data_matrix.npy', matrix)

# Load the saved matrix from the file
loaded_matrix = np.load('data_matrix.npy')

print("Original matrix:")
print(matrix)

print("Loaded matrix:")
```

```python
print(loaded_matrix)

# Verify if the original matrix and the loaded matrix are the same
if np.array_equal(matrix, loaded_matrix):
    print("The original matrix and the loaded matrix are the same.")
else:
    print("The original matrix and the loaded matrix are different.")
```

```
Original matrix:
[[0.50434109 0.767648   0.94990819 0.33829727]
 [0.08884936 0.1278159  0.39435415 0.89734447]
 [0.80202846 0.51020349 0.30299424 0.47010542]
 [0.41747688 0.1773442  0.31360666 0.97802798]]
Loaded matrix:
[[0.50434109 0.767648   0.94990819 0.33829727]
 [0.08884936 0.1278159  0.39435415 0.89734447]
 [0.80202846 0.51020349 0.30299424 0.47010542]
 [0.41747688 0.1773442  0.31360666 0.97802798]]
The original matrix and the loaded matrix are the same.
```

```python
np.savetxt("data_matrix.csv", matrix, delimiter=",")
loaded_csv = np.loadtxt("data_matrix.csv", delimiter=",")

a = np.array([11, 22])
b = np.array([2, 2])

if (a > b).any():
    print("At least one element in a is greater than b")

if (a > b).all():
    print("All elements in a are greater than b")
```

```
At least one element in a is greater than b
All elements in a are greater than b
```

```python
import numpy as np

a = np.array([1, 3])
b = np.array([2, 2])

# Element-wise comparison mask
mask = a > b
print("Comparison mask (a > b):", mask)
# → [False False]

# Extracting the values in a that satisfy the condition
greater_elements = a[mask]
print("Elements in a that are > b:", greater_elements)
# → []  (an empty array, since none match)
```

```
Comparison mask (a > b): [False  True]
Elements in a that are > b: [3]
```

```
mask = (a > b) | (a == 3)
print("Custom filtered elements:", a[mask])
```

```
Custom filtered elements: [3]
```

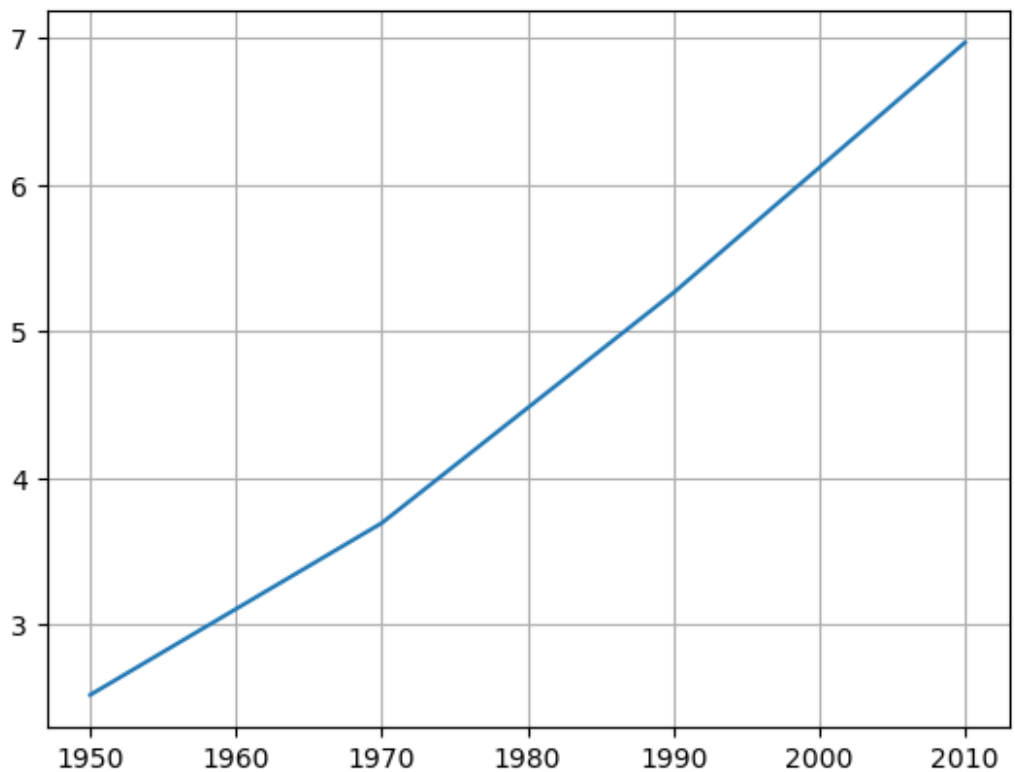
```
# a = np.array([1, 2])
# b = np.array([2, 2])
# print(a)
# print(b)
# if a>b:
#     a
```

Matplotlib

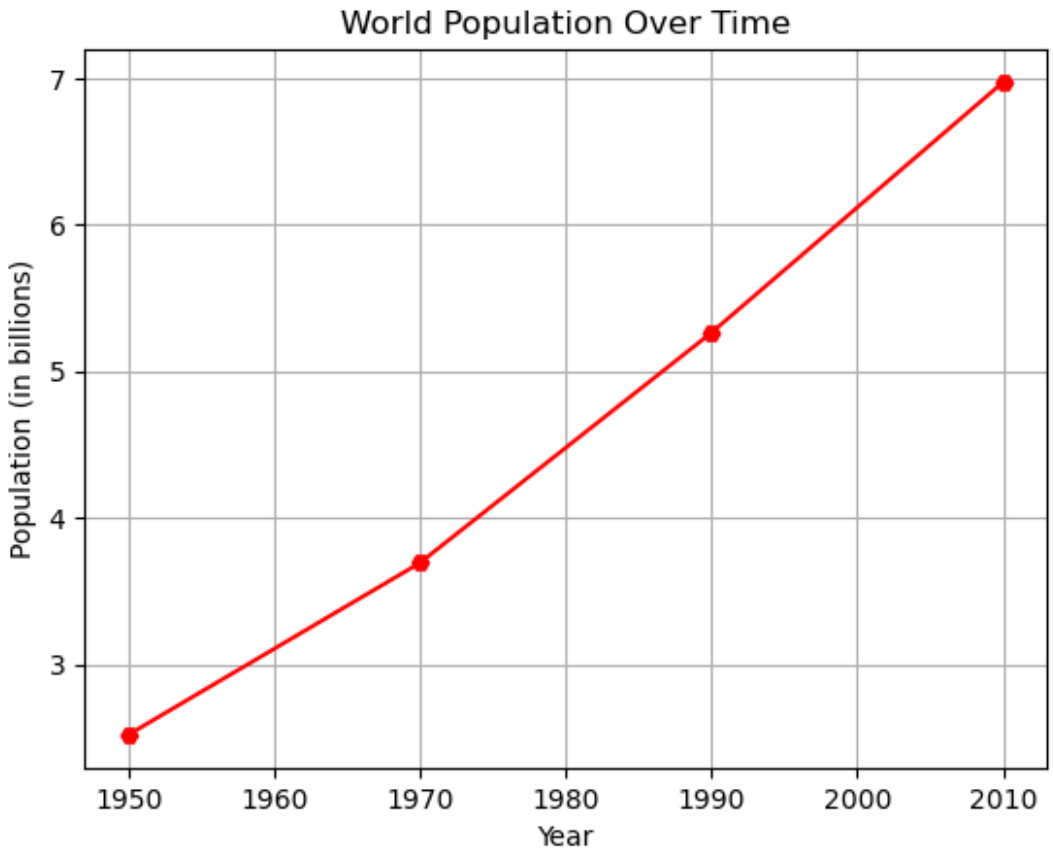
```
# !pip install --upgrade matplotlib ipykernel traitlets pybind11

# !pip install "numpy<2"
```

Basic line plot

```
import matplotlib.pyplot as plt

# Basic line plot
year = [1950, 1970, 1990, 2010]  # X-axis values representing years
pop = [2.519, 3.692, 5.263, 6.972]  # Y-axis values representing
population in billions

plt.plot(year, pop)  # Line plot connecting the points
plt.grid(True)  # Display grid for better readability
plt.show()  # Render the plot
```



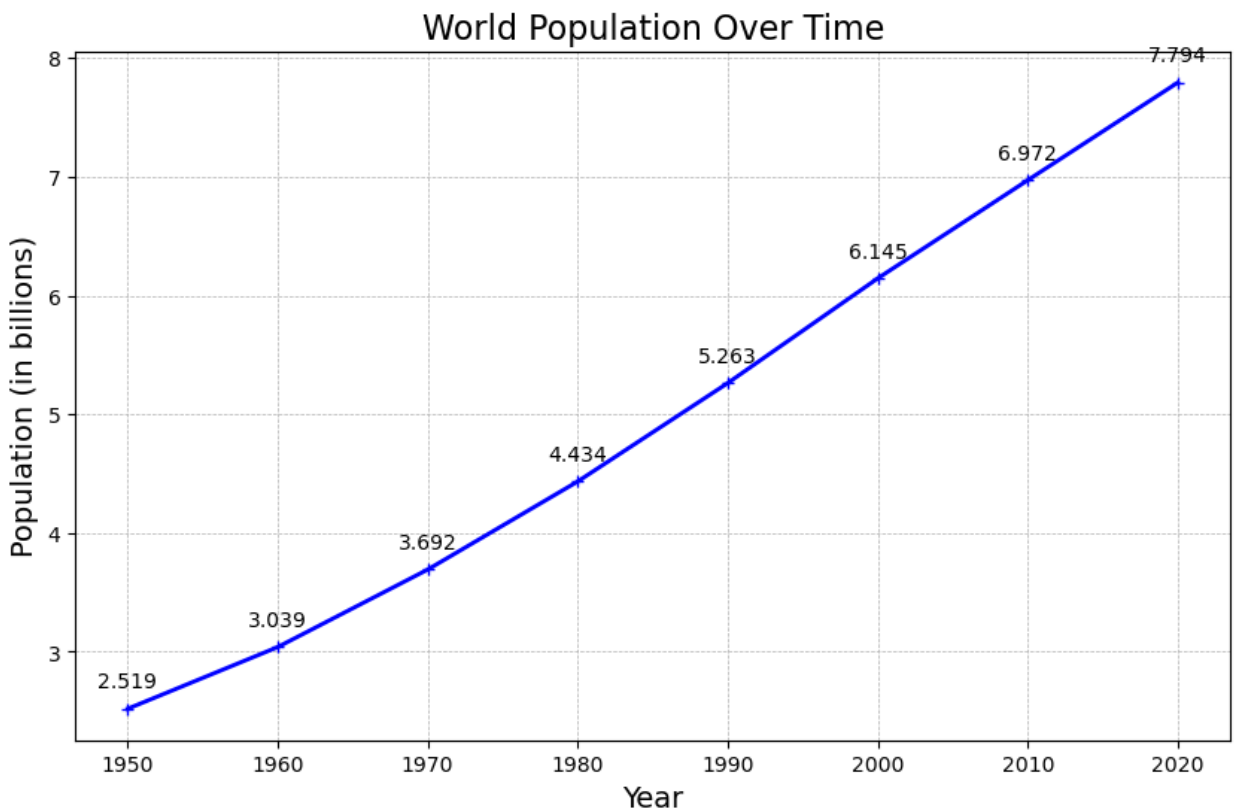

```
## Enhanced version with labels and custom style
plt.plot(year, pop, marker='H', color='red')  # 'H' marker and red
line
plt.grid(True)
plt.xlabel('Year')  # X-axis label
plt.ylabel('Population (in billions)')  # Y-axis label
plt.title('World Population Over Time')  # Title
plt.show()
```



Enhanced version with labels and custom style

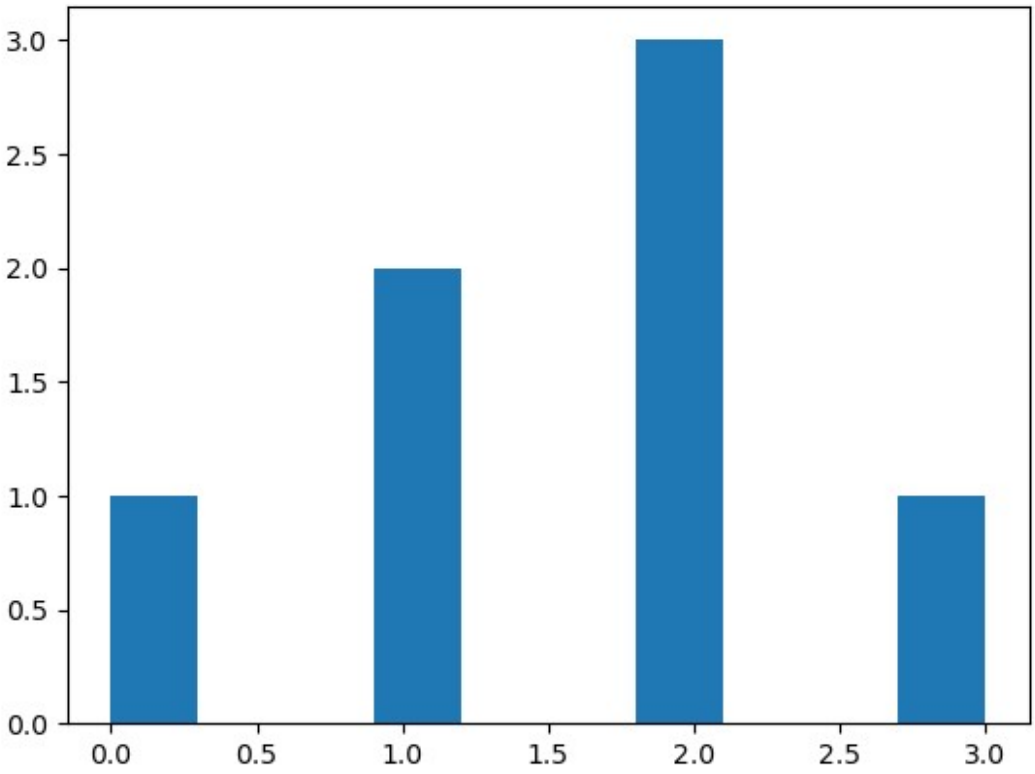
```
import matplotlib.pyplot as plt

# Data
year = [1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020]
pop = [2.519, 3.039, 3.692, 4.434, 5.263, 6.145, 6.972, 7.794]

# Plotting the data
plt.figure(figsize=(10, 6))  # Set the figure size
plt.plot(year, pop, marker='+', color='blue', linestyle='-',
linewidth=2, markersize=6)  # Added marker, color, linestyle,
linewidth, and markersize

# Adding grid, labels, and title
plt.grid(True, linestyle='--', linewidth=0.5)
plt.xlabel('Year', fontsize=14)
plt.ylabel('Population (in billions)', fontsize=14)
plt.title('World Population Over Time', fontsize=16)

# Adding annotations for data points
for i, txt in enumerate(pop):
    plt.annotate(f'{txt}', (year[i], pop[i]), textcoords="offset
```

```
points", xytext=(0,10), ha='center')
# This loop adds annotations (text labels) above each data point.
# enumerate(pop): Loops through population values along with their
indices.
# annotate(f'{txt}', ...): Places the population value as a label
above the point.
# xytext=(0,10): Shifts the label 10 pixels upward.
# ha='center': Horizontally centers the text above the point.
# Show plot
plt.show()
```

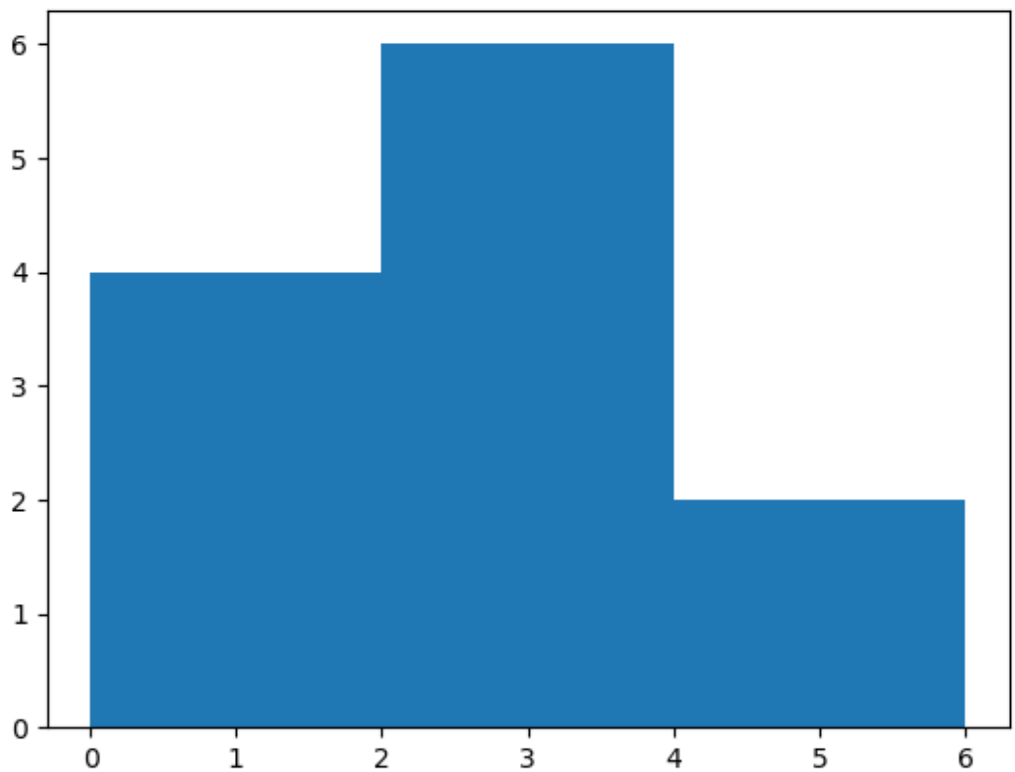


Histogram

```
values = [0,1,1,2,2,2,3]
plt.hist(values)
plt.show()
```



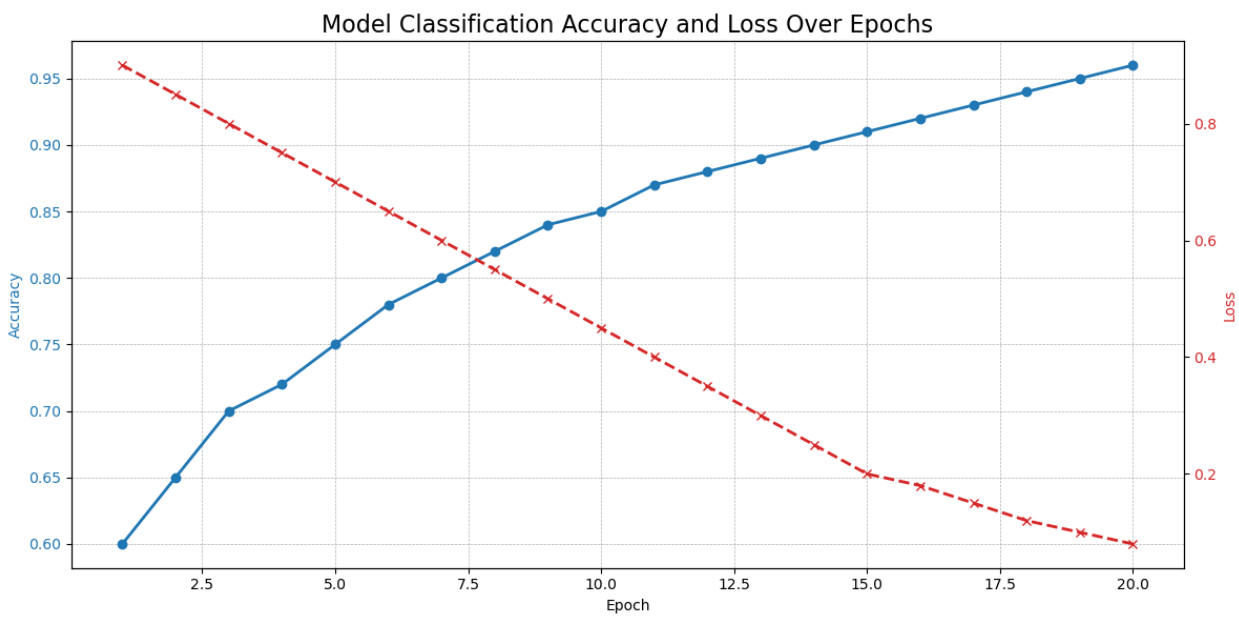

```
values = [0,0.6,1.4,1.6,2.2,2.5,2.6,3.2,3.5,3.9,4.2,6]
plt.hist(values, bins=3)
plt.show()
```



Dual Axis Plot (Accuracy vs Loss over Epochs)

```
import matplotlib.pyplot as plt

# Example data for classification accuracy and loss
epochs = list(range(1, 21))
accuracy = [0.6, 0.65, 0.7, 0.72, 0.75, 0.78, 0.8, 0.82, 0.84, 0.85,
0.87, 0.88, 0.89, 0.9, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96]
loss = [0.9, 0.85, 0.8, 0.75, 0.7, 0.65, 0.6, 0.55, 0.5, 0.45, 0.4,
0.35, 0.3, 0.25, 0.2, 0.18, 0.15, 0.12, 0.1, 0.08]

# Create a figure and axis
fig, ax1 = plt.subplots(figsize=(12, 6))

# Plot accuracy
color = 'tab:blue'
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy', color=color)
ax1.plot(epochs, accuracy, marker='o', color=color, linestyle='-',
linewidth=2, markersize=6, label='Accuracy')
ax1.tick_params(axis='y', labelcolor=color)

# Create a second y-axis for the loss
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Loss', color=color)
```

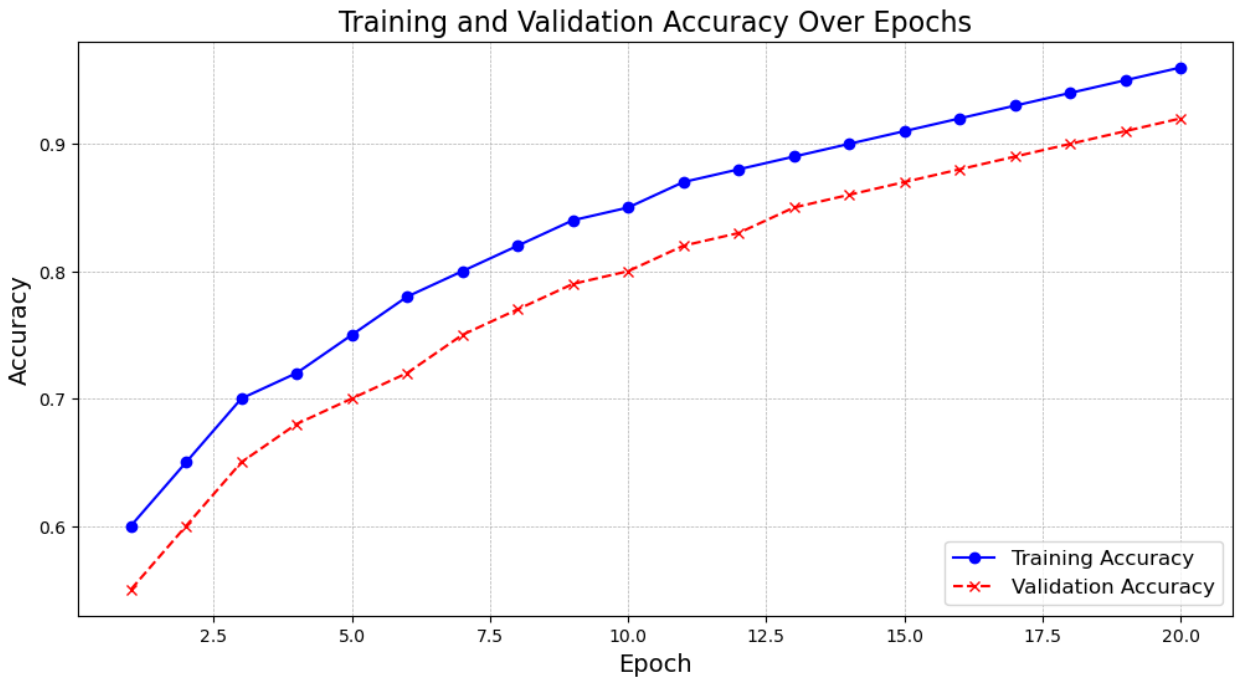
```
ax2.plot(epochs, loss, marker='x', color=color, linestyle='--',
linewidth=2, markersize=6, label='Loss')
ax2.tick_params(axis='y', labelcolor=color)

# Add title and grid
plt.title('Model Classification Accuracy and Loss Over Epochs',
fontsize=16)
fig.tight_layout()
ax1.grid(True, linestyle='--', linewidth=0.5)

# Show the plot
plt.show()
```



Train vs Validation Accuracy Plot

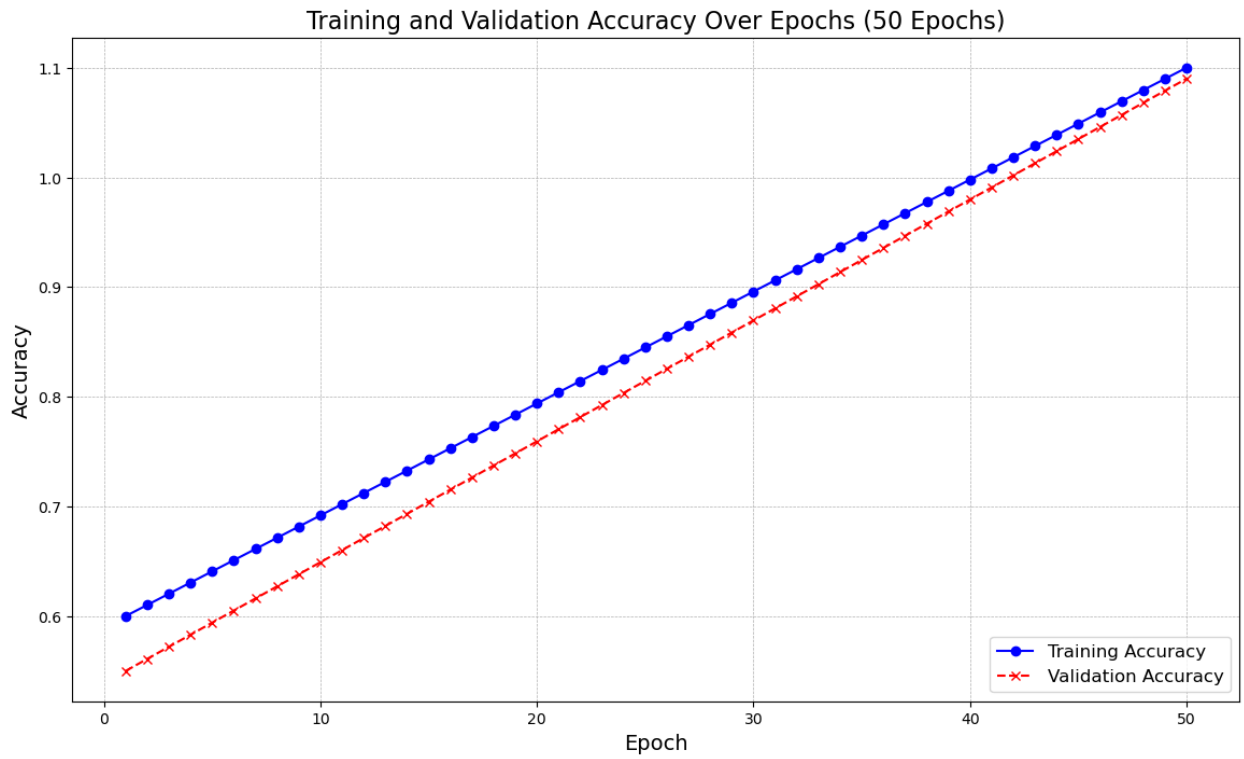
```
# --- Train vs Validation Accuracy Plot ---
train_accuracy = [0.6, 0.65, 0.7, 0.72, 0.75, 0.78, 0.8, 0.82, 0.84,
0.85,
                  0.87, 0.88, 0.89, 0.9, 0.91, 0.92, 0.93, 0.94, 0.95,
0.96]
val_accuracy = [0.55, 0.6, 0.65, 0.68, 0.7, 0.72, 0.75, 0.77, 0.79,
0.8,
                0.82, 0.83, 0.85, 0.86, 0.87, 0.88, 0.89, 0.9, 0.91,
0.92]

plt.figure(figsize=(12, 6))
plt.plot(epochs, train_accuracy, marker='o', color='blue',
linestyle='-', label='Training Accuracy')
plt.plot(epochs, val_accuracy, marker='x', color='red',
linestyle='--', label='Validation Accuracy')
```

```
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs', fontsize=16)
plt.legend(loc='lower right', fontsize=12)
plt.grid(True, linestyle='--', linewidth=0.5)
plt.show()
```



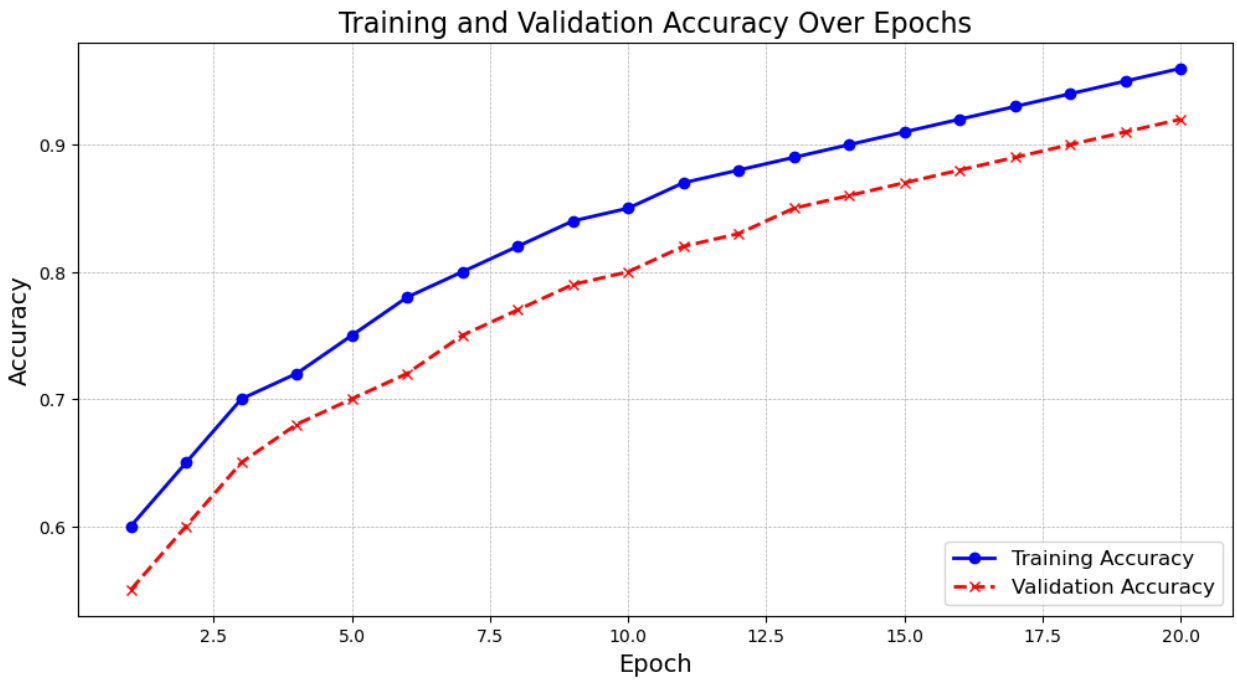
Longer 50-Epoch Version

```
# --- Longer 50-Epoch Version ---
epochs = list(range(1, 51))
train_accuracy = np.linspace(0.6, 1.1, 50)
val_accuracy = np.linspace(0.55, 1.09, 50)

plt.figure(figsize=(14, 8))
plt.plot(epochs, train_accuracy, marker='o', color='blue',
label='Training Accuracy')
plt.plot(epochs, val_accuracy, marker='x', color='red',
linestyle='--', label='Validation Accuracy')
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs (50 Epochs)',
fontsize=16)
plt.legend(loc='lower right', fontsize=12)
plt.grid(True, linestyle='--', linewidth=0.5)
plt.show()
```



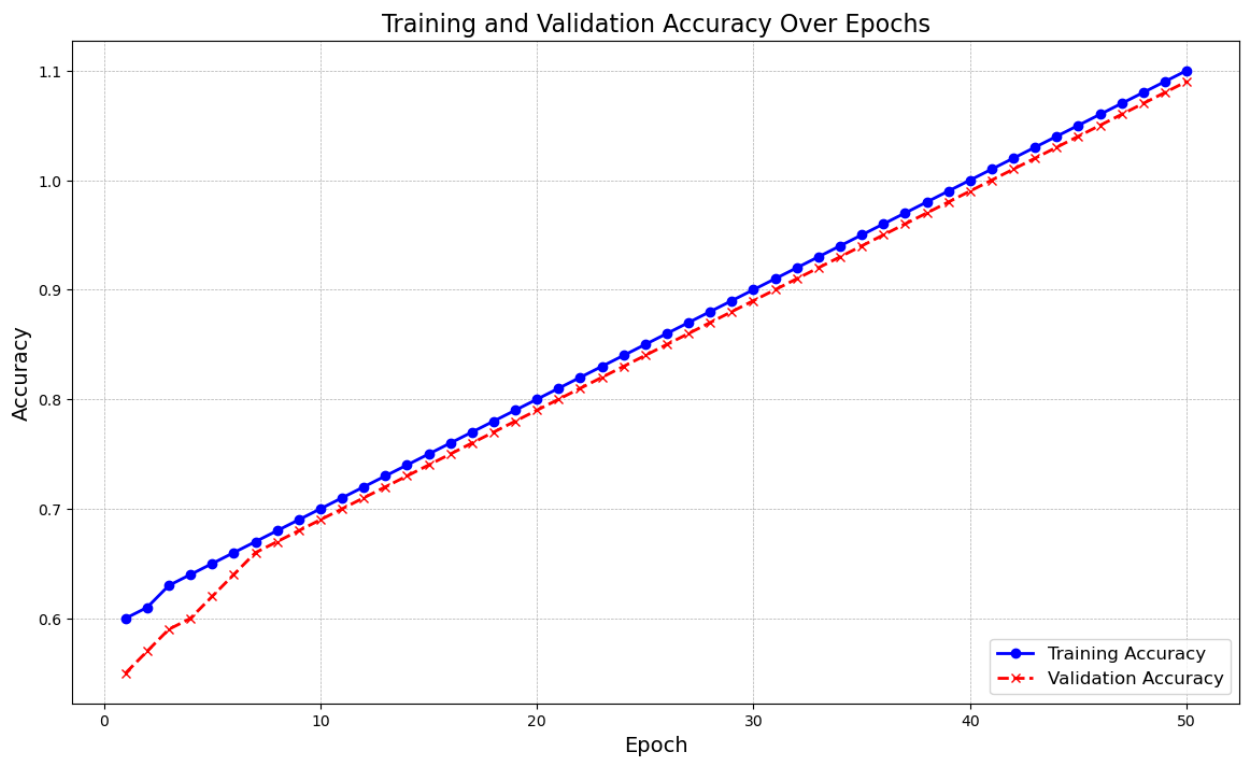

```python
import matplotlib.pyplot as plt

# Example data for training and validation accuracy
epochs = list(range(1, 21))
train_accuracy = [0.6, 0.65, 0.7, 0.72, 0.75, 0.78, 0.8, 0.82, 0.84,
0.85, 0.87, 0.88, 0.89, 0.9, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96]
val_accuracy = [0.55, 0.6, 0.65, 0.68, 0.7, 0.72, 0.75, 0.77, 0.79,
0.8, 0.82, 0.83, 0.85, 0.86, 0.87, 0.88, 0.89, 0.9, 0.91, 0.92]

# Create a figure and axis
plt.figure(figsize=(12, 6))

# Plot training accuracy
plt.plot(epochs, train_accuracy, marker='o', color='blue',
linestyle='-', linewidth=2, markersize=6, label='Training Accuracy')

# Plot validation accuracy
plt.plot(epochs, val_accuracy, marker='x', color='red',
linestyle='--', linewidth=2, markersize=6, label='Validation
Accuracy')

# Add labels, title, and legend
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs', fontsize=16)
plt.legend(loc='lower right', fontsize=12)
```

```
# Add grid
plt.grid(True, linestyle='--', linewidth=0.5)

# Show plot
plt.show()
```



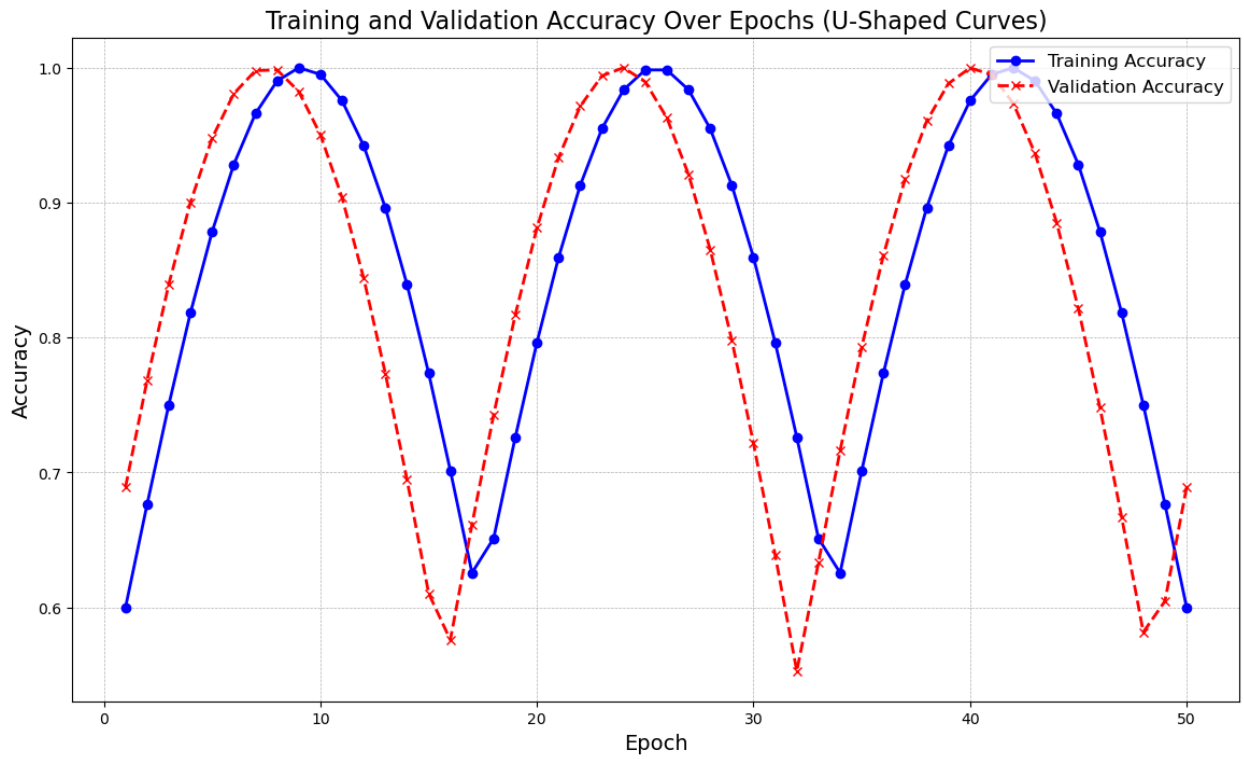

```
import matplotlib.pyplot as plt

# Example data for training and validation accuracy
epochs = list(range(1, 51))
train_accuracy = [0.6, 0.61, 0.63, 0.64, 0.65, 0.66, 0.67, 0.68, 0.69,
0.7,
                  0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77, 0.78,
0.79, 0.8,
                  0.81, 0.82, 0.83, 0.84, 0.85, 0.86, 0.87, 0.88,
0.89, 0.9,
                  0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98,
0.99, 1.0,
                  1.01, 1.02, 1.03, 1.04, 1.05, 1.06, 1.07, 1.08,
1.09, 1.1]
val_accuracy = [0.55, 0.57, 0.59, 0.6, 0.62, 0.64, 0.66, 0.67, 0.68,
0.69,
                0.7, 0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77, 0.78,
0.79,
                0.8, 0.81, 0.82, 0.83, 0.84, 0.85, 0.86, 0.87, 0.88,
0.89,
                0.9, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98,
0.99,
```

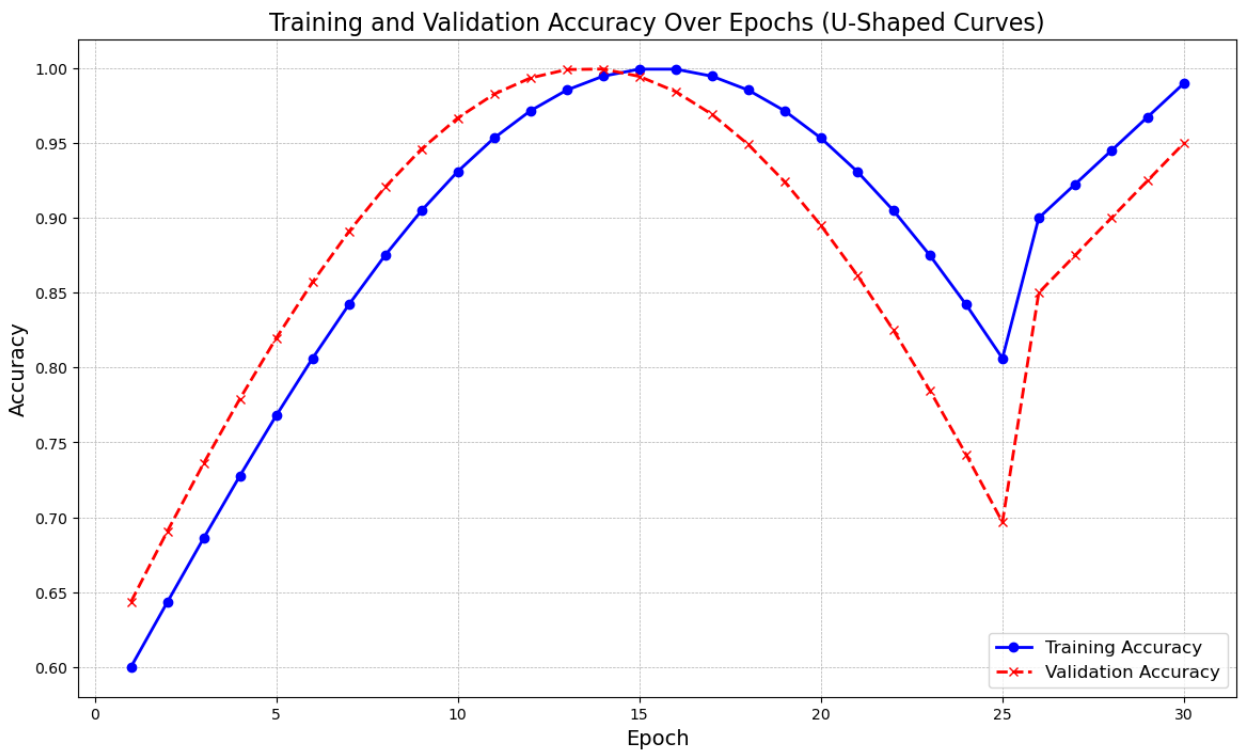
```
                 1.0, 1.01, 1.02, 1.03, 1.04, 1.05, 1.06, 1.07, 1.08,
1.09]

# Create a figure and axis
plt.figure(figsize=(14, 8))

# Plot training accuracy
plt.plot(epochs, train_accuracy, marker='o', color='blue',
linestyle='-', linewidth=2, markersize=6, label='Training Accuracy')

# Plot validation accuracy
plt.plot(epochs, val_accuracy, marker='x', color='red',
linestyle='--', linewidth=2, markersize=6, label='Validation
Accuracy')

# Add labels, title, and legend
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs', fontsize=16)
plt.legend(loc='lower right', fontsize=12)

# Add grid
plt.grid(True, linestyle='--', linewidth=0.5)

# Show plot
plt.show()
```



U-Shaped Curve

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for U-shaped training and validation accuracy
epochs = np.arange(1, 51)
train_accuracy = 0.6 + 0.4 * np.abs(np.sin(np.linspace(0, 3 * np.pi,
50)))
val_accuracy = 0.55 + 0.45 * np.abs(np.sin(np.linspace(0, 3 * np.pi,
50) + np.pi / 10))

# Create a figure and axis
plt.figure(figsize=(14, 8))

# Plot training accuracy
plt.plot(epochs, train_accuracy, marker='o', color='blue',
linestyle='-', linewidth=2, markersize=6, label='Training Accuracy')

# Plot validation accuracy
plt.plot(epochs, val_accuracy, marker='x', color='red',
linestyle='--', linewidth=2, markersize=6, label='Validation
Accuracy')

# Add labels, title, and legend
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs (U-Shaped
Curves)', fontsize=16)
plt.legend(loc='upper right', fontsize=12)

# Add grid
plt.grid(True, linestyle='--', linewidth=0.5)

# Show plot
plt.show()
```




```python
import matplotlib.pyplot as plt
import numpy as np

# Set number of epochs
epochs = np.arange(1, 31)

# Generate U-shaped curves for training and validation accuracy (30 points)
train_accuracy = 0.6 + 0.4 * np.abs(np.sin(np.linspace(0, np.pi, 30)))
val_accuracy = 0.55 + 0.45 * np.abs(np.sin(np.linspace(0, np.pi, 30) + np.pi / 15))

# Ensure accuracy reaches close to 100% by the end
train_accuracy[-5:] = np.linspace(0.9, 0.99, 5)
val_accuracy[-5:] = np.linspace(0.85, 0.95, 5)

# Create a figure and axis
plt.figure(figsize=(14, 8))

# Plot training accuracy
plt.plot(epochs, train_accuracy, marker='o', color='blue', linestyle='-', linewidth=2, markersize=6, label='Training Accuracy')

# Plot validation accuracy
plt.plot(epochs, val_accuracy, marker='x', color='red', linestyle='--', linewidth=2, markersize=6, label='Validation Accuracy')
```

```
# Add labels, title, and legend
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Accuracy', fontsize=14)
plt.title('Training and Validation Accuracy Over Epochs (U-Shaped
Curves)', fontsize=16)
plt.legend(loc='lower right', fontsize=12)

# Add grid
plt.grid(True, linestyle='--', linewidth=0.5)

# Show plot
plt.show()
```